

Simulacro Parcial II

Estimados:

Este es el primer simulacro y presenta un nivel de dificultad elevado, ya que integra todos los temas vistos hasta el momento. Su resolución requiere (y demuestra) un manejo sólido de Angular.

No se preocupen si no logran completarlo a la primera: es algo totalmente esperado. Justamente por eso, estas semanas estarán dedicadas exclusivamente a la práctica. El objetivo de este simulacro es que puedan repasar cada uno de los temas, profundizar en los conceptos que lo requieran y avanzar de manera gradual en la comprensión del framework.

Una vez finalizado este trabajo, contarán con la preparación necesaria para afrontar el segundo parcial.

Avancen paso a paso, sin apuro. ¡Muchos éxitos!

1. Creación del proyecto

- Crear un proyecto Angular nuevo con el nombre gestion-crud.

2. Modelo de datos

- Crear una carpeta models y definir allí una interfaz/clase que represente el tipo de dato con el que trabajará la aplicación (ejemplo: Movie, Book, Product, etc.).
- El modelo debe contener al menos 3 atributos obligatorios (ejemplo: id, título, anio).

3. Json Server

- Configurar json-server para simular la persistencia de datos.
- Crear un archivo db.json que contenga un array con algunos registros iniciales del modelo elegido.
- Verificar que el servidor responda en la ruta `http://localhost:3000/`.

4. Estructura de la aplicación (Pages)

- Crear las siguientes pages/componentes principales, todas con header y footer visibles:

- HomePage: página de inicio, con una breve descripción del sitio.
 - ListPage: muestra la lista de todos los elementos del modelo (traídos desde json-server con HttpClient).
 - FormPage: formulario para crear un nuevo registro.
 - FormEditPage: formulario para editar un registro existente.
-

5. Routing

- Configurar el enrutamiento para que cada page tenga su propia ruta.
 - Agregar rutas paramétricas para acceder al detalle de un elemento en un componente aparte (por ejemplo, details/:id).
-

6. Formularios reactivos y validaciones

- Todos los formularios deben ser reactivos (usando FormGroup y FormControl, o FormBuilder).
 - Aplicar validaciones con Validators, como por ejemplo:
 - Campo obligatorio.
 - Longitud mínima/máxima.
 - Patrón (ej: que el año sea un número de 4 dígitos).
 - Mostrar mensajes de error debajo de los campos cuando no se cumplan las validaciones.
-

7. CRUD completo con servicio HTTP

- Crear un Service encargado de realizar todas las peticiones HTTP (GET, POST, PUT, DELETE) al json-server.
 - Implementar en la aplicación las operaciones:
 - Post: Desde FormPage, crear un registro nuevo mediante un formulario reactivo.
 - Get: Mostrar todos los registros en ListPage. Además, al hacer clic en un ítem de la lista, navegar a una DetailsPage que muestre toda la información del objeto mediante una ruta paramétrica.
 - Put: Desde FormEditPage, cargar los datos existentes en el formulario (según su id), permitir la edición y guardar cambios.
 - Delete: Desde ListPage, eliminar un registro mediante un botón.
-

8. ngOnInit

- Usar ngOnInit en los componentes donde corresponda.